

OpenAI Agents SDK: From Zero to Your First Tool-Using Agent

A hands-on guide for beginners

This guide takes you from nothing to a working AI agent, in the order you should actually learn it:

1. **Setup** — get a project ready.
2. **OpenAI key integration** — the simplest possible agent (SDK's native, default provider).
3. **Gemini key integration** — swap in a different LLM provider.
4. **Runner and asyncio** — how an agent actually executes, and why it's async under the hood.
5. **Model configuration** — the three levels (Agent / Run / Global) for choosing which LLM to use.
6. **Basic tool use** — give your agent the ability to call Python functions with `@function_tool`.
7. **Model settings** — turn the model's knobs: temperature, tool choice, answer length.
8. **Local context** — hand your tools data about the user that the LLM never sees.
9. **Dynamic instructions** — build the system prompt at runtime instead of hardcoding it.
10. **Agent cloning** — spin specialised variants off one base agent.

Everything here is copy-paste ready. Type the code yourself rather than pasting — it sticks better.

Part 0 — Setup

You need **Python 3.13+** and **uv** (a fast Python package manager). If you don't have **uv**:

```
pip install uv
```

Create a project:

```
uv init hello-agent
cd hello-agent
uv add openai-agents python-dotenv
```

Create a **.env** file in the project root — this is where your API keys live. **Never commit this file.**

```
OPENAI_API_KEY=
GEMINI_API_KEY=
```

You'll fill these in as you go through Parts 1 and 2.

Part 1 — OpenAI Key Integration (the default provider)

The Agents SDK is built by OpenAI, so **OpenAI is the default provider** — no custom client setup required. You just need an API key.

Get a key

1. Go to <https://platform.openai.com/api-keys> and create a key.
2. Paste it into `.env`:

```
OPENAI_API_KEY=sk-...
```

Write the agent

Create `main.py`:

```
from agents import Agent, Runner
from dotenv import load_dotenv, find_dotenv

# Loads .env so OPENAI_API_KEY is available as an environment variable
load_dotenv(find_dotenv())

# The SDK finds OPENAI_API_KEY automatically — no client wiring needed
agent = Agent(
    name="Assistant",
    instructions="You are a helpful assistant.",
)

result = Runner.run_sync(agent, "Write a haiku about recursion in programming.")
print(result.final_output)
```

Run it:

```
uv run main.py
```

That's the whole integration. Three things happened:

- `Agent(...)` — defined *who* the agent is (a name + a system prompt via `instructions`).
- `Runner.run_sync(...)` — actually sent the prompt to the LLM and waited for the result.
- `result.final_output` — the agent's final text response.

Note: If you don't pass a `model=`, the Agent uses the SDK's built-in default OpenAI model. You'll learn to override this explicitly in Part 4.

Part 2 — Gemini Key Integration (a different provider)

The Agents SDK only *ships* with OpenAI wiring, but it can talk to **any provider that exposes an OpenAI-compatible Chat Completions API** — and Google's Gemini API does exactly that.

Get a key

1. Go to <https://ai.google.dev/gemini-api/docs/api-key> and create a Gemini API key.
2. Paste it into `.env`:

```
GEMINI_API_KEY=...
```

Write the agent

```
from agents import Agent, Runner, OpenAIChatCompletionsModel, AsyncOpenAI
import os
from dotenv import load_dotenv, find_dotenv

load_dotenv(find_dotenv())

# 1. Point an OpenAI-compatible client at Gemini's endpoint instead of OpenAI's
external_client = AsyncOpenAI(
    api_key=os.getenv("GEMINI_API_KEY"),
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)

# 2. Wrap it as a Chat Completions model the SDK understands
llm_model = OpenAIChatCompletionsModel(
    model="gemini-2.5-flash",
    openai_client=external_client,
)

# 3. Hand that model to the Agent instead of using the default
agent = Agent(name="Assistant", model=llm_model)

result = Runner.run_sync(agent, "Welcome and motivate me to learn Agentic AI.")
print("AGENT RESPONSE:", result.final_output)
```

Why this works

- `AsyncOpenAI` is just an HTTP client shaped for OpenAI's API contract. Gemini implements that same contract at a different `base_url`, so the same client class works — you're not using any Google-specific SDK.
- `OpenAIChatCompletionsModel` tells the Agent "call this client using the Chat Completions format, with this specific model name."
- Everything else — `Agent`, `Runner`, `.final_output` — is identical to Part 1. **This is the whole point of the SDK's design:** the provider is just a pluggable `model`, the rest of your agent code never changes.

Part 3 — Understanding `Runner` and `asyncio`

You've used `Runner.run_sync(...)` twice without asking what it does. Time to open that up.

Why agents are async in the first place

Every call an agent makes to an LLM is a **network request** — it goes out over HTTP and the program sits waiting for a response. That's *I/O-bound* waiting, not CPU work. Python's `asyncio` exists so a program can do other useful things (or run several LLM calls concurrently) instead of blocking dead while it waits on the network. The Agents SDK is written natively with `async/await` for this reason.

The three ways to run an agent

Method	Signature	When to use it
<code>Runner.run(...)</code>	<code>async def run(...)</code> — must be <code>awaited</code>	Inside an <code>async def</code> function — real applications, servers, concurrent agents
<code>Runner.run_sync(...)</code>	plain function	Scripts, notebooks, quick tests — internally starts an event loop and blocks for you
<code>Runner.run_streamed(...)</code>	returns a streaming result	When you want to show tokens/events as they arrive, instead of waiting for the whole answer

The `async` version, properly

```
import asyncio
from agents import Agent, Runner

agent = Agent(name="Assistant", instructions="You only respond in haikus.")

async def main():
    result = await Runner.run(agent, "Tell me about recursion in programming.")
    print(result.final_output)

if __name__ == "__main__":
    asyncio.run(main())
```

Breaking this down:

- `async def main()` — declares a coroutine, a function that can pause and resume instead of blocking.
- `await Runner.run(...)` — pause `main()` here until the LLM responds, without freezing the whole program.
- `asyncio.run(main())` — creates the event loop, runs `main()` to completion, and shuts the loop down. This is the standard entry point for any async Python script.

`Runner.run_sync(agent, "...")` you used in Parts 1–2 is a convenience wrapper that does exactly `asyncio.run(Runner.run(agent, "..."))` under the hood. **Use `run_sync` for quick scripts. Use `run` inside real async apps** — e.g. a FastAPI endpoint, or when you want to fire off multiple agents at once with `asyncio.gather(...)`.

Try it: run two agents concurrently

```
import asyncio
from agents import Agent, Runner

agent = Agent(name="Assistant", instructions="Answer in one short sentence.")

async def main():
    results = await asyncio.gather(
        Runner.run(agent, "What is Python?"),
        Runner.run(agent, "What is an AI agent?"),
    )
    for r in results:
        print(r.final_output)

asyncio.run(main())
```

Both prompts are in flight to the LLM provider at the same time — this is only possible because `Runner.run` is async.

Part 4 — Model Configuration: Agent, Run, and Global Levels

You already did **Agent-level** configuration in Part 2 (passing `model=` straight into `Agent(...)`). That's the SDK's default provider is OpenAI; you can override it at three different scopes.

We will always use Agent-level configuration in this course — it lets each agent in a multi-agent system use whichever model fits its job (e.g., a cheap fast model for routing, a stronger model for reasoning).

1. Agent Level — override per agent

```
from agents import Agent, OpenAIChatCompletionsModel, AsyncOpenAI, Runner, set_tracing_disabled

client = AsyncOpenAI(
    api_key=gemini_api_key,
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)
set_tracing_disabled(disabled=True) # OpenAI's tracing dashboard expects an OpenAI key; disable

agent = Agent(
    name="Assistant",
    instructions="You only respond in haikus.",
    model=OpenAIChatCompletionsModel(model="gemini-2.0-flash", openai_client=client),
)
```

Different agents in the same app can each have their own `model=`.

2. Run Level — override for one call

```
from agents.run import RunConfig

config = RunConfig(
    model=OpenAIChatCompletionsModel(model="gemini-2.0-flash", openai_client=external_client),
    model_provider=external_client,
    tracing_disabled=True,
)

agent = Agent(name="Assistant", instructions="You are a helpful assistant")
result = Runner.run_sync(agent, "Hello, how are you.", run_config=config)
```

The agent itself is unconfigured; the override applies only to this specific `Runner.run_sync(...)` call — useful for A/B testing a model without touching agent definitions.

3. Global Level — override for the whole process

```
from agents import set_default_openai_client, set_default_openai_api, set_tracing_disabled

set_tracing_disabled(True)
set_default_openai_api("chat_completions")
set_default_openai_client(external_client)

agent = Agent(name="Assistant", instructions="You are a helpful assistant", model="gemini-2.0-")
```

Every agent created afterward uses this client by default, unless it — or its run — overrides it locally. Agent-level and Run-level configuration both win over Global.

Precedence, most specific wins: Agent level > Run level > Global level.

Part 5 — Basic Tool Use with `@function_tool`

So far your agent can only talk. A **tool** lets it *do* something — call a Python function, hit an API, query a database — and use the result to answer better.

The core idea

You write a normal Python function. The `@function_tool` decorator turns it into something the LLM can call:

- The **function name** becomes the tool's name.
- The **docstring** becomes the tool's description (this is what tells the LLM *when* to use it).
- The **type hints** are used to auto-generate the JSON schema for the tool's parameters.

```
from agents import Agent, Runner, function_tool

@function_tool
def get_weather(city: str) -> str:
    """Get the current weather for a given city."""
    return f"The weather in {city} is sunny and 25°C."

agent = Agent(
    name="Weather Assistant",
    instructions="You are a helpful assistant. Use tools when they help you answer.",
    tools=[get_weather],
)

result = Runner.run_sync(agent, "What's the weather like in Karachi?")
print(result.final_output)
```

What actually happens when you run this

1. `Runner.run_sync` sends your prompt **and** the tool's schema (name, description, parameters) to the LLM.
2. The LLM decides the question needs live data it doesn't have, and responds with "call `get_weather` with `city="Karachi"`" instead of a text answer.
3. The SDK intercepts that, **runs your actual Python function** with those arguments, and sends the return value back to the LLM as a new message.
4. The LLM reads the tool's result and writes the final natural-language answer.
5. `Runner.run_sync` only returns to you once this whole loop is finished — `result.final_output` is the final answer, tool calls included.

You can register multiple tools; the agent decides on its own which (if any) to call, and can even chain several tool calls in one turn.

Try it: add a second tool

```
@function_tool
def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b

agent = Agent(
    name="Assistant",
    instructions="Use tools when they help you answer.",
    tools=[get_weather, add],
)

result = Runner.run_sync(agent, "What's 42 plus 58, and what's the weather in Lahore?")
print(result.final_output)
```

Watch the model correctly call both tools in the same turn and combine both results into one answer.

Beyond `@function_tool`

`@function_tool` is for **your own** Python functions. The SDK also ships **hosted/built-in OpenAI tools** — like `WebSearchTool` and `FileSearchTool` — that run on OpenAI's servers instead of your machine. Those are out of scope for this guide; `@function_tool` is the one you'll use 90% of the time, and it's the foundation everything else builds on.

Part 6 — Model Settings: Temperature, Tool Choice, and Length

Part 4 decided **which** model runs your agent. `ModelSettings` decides **how** that model behaves once it's running — how creative it is, whether it's allowed to reach for a tool, and how long its answer can get. Same agent, same prompt, different output.

The three settings to learn first

- **temperature** — the creativity dial. Low keeps answers focused and repeatable; high makes them varied and surprising.
- **tool_choice** — whether the model may, must, or must not call a tool.
- **max_tokens** — a hard ceiling on how long the answer can be.

You pass them as one object to the Agent:

```

from agents import Agent, Runner, ModelSettings

question = "Tell me about AI in 2 sentences."

for temperature in (0.1, 1.9):
    agent = Agent(
        name="Assistant",
        instructions="You are a helpful assistant.",
        model=llm_model,
        model_settings=ModelSettings(temperature=temperature),
    )
    print(f"\ntemperature={temperature}:")
    print(Runner.run_sync(agent, question).final_output)

```

Run it twice. The **0.1** answer will barely change between runs; the **1.9** answer will come out different every time. Use low temperature for maths, facts and instruction-following; high for brainstorming and story writing.

Note: The usable range depends on the provider. Gemini accepts a **temperature** up to **2.0**, so **1.9** is a legal (and very loose) setting here.

Forcing — or forbidding — tool use

tool_choice overrides the model's own judgement about the tools you registered in Part 5.

```

from agents import function_tool

@function_tool
def calculate_area(length: float, width: float) -> str:
    """Calculate the area of a rectangle."""
    return f"Area = {length} × {width} = {length * width} square units"

question = "What's the area of a 5x3 rectangle?"

for choice in ("auto", "required", "none"):
    agent = Agent(
        name=f"{choice} agent",
        instructions="You are a helpful assistant.",
        model=llm_model,
        tools=[calculate_area],
        model_settings=ModelSettings(tool_choice=choice),
    )
    print(f"\ntool_choice={choice}:")
    print(Runner.run_sync(agent, question).final_output)

```

- **"auto"** — the model decides. This is the default, and what Part 5 was already doing.
- **"required"** — the model **must** call a tool before it may answer. Useful when guessing is worse than failing (calculations, lookups).

- **"none"** — the model still sees the tool's schema but is forbidden from calling it. It will answer from its own head, and you'll see it do the arithmetic itself.

Try it: cap the answer length

```
brief_agent = Agent(  
    name="Brief Assistant",  
    instructions="You are a helpful assistant.",  
    model=llm_model,  
    model_settings=ModelSettings(temperature=0.2, max_tokens=100),  
)
```

max_tokens cuts the response off at the limit — it does **not** ask the model to be concise. If the answer needs more room than you gave it, it stops mid-sentence. Say "answer in two sentences" in the **instructions** when you want brevity; use **max_tokens** when you want a cost ceiling.

The rest of the knobs

ModelSettings also carries **parallel_tool_calls** (let the agent fire several tools at once instead of one at a time), **top_p**, **frequency_penalty** and **presence_penalty**. Leave all of them at their defaults until you have a specific reason not to — changing one setting at a time is the only way to tell what actually helped.

Part 7 — Local Context: Data Your Tools See and the LLM Doesn't

Your tools so far have been self-contained: everything they needed arrived as an argument from the LLM. Real tools need things the LLM shouldn't be inventing — who the logged-in user is, their ID, a database handle, a logger. **Local context** is how you pass those in.

The core idea

You define a plain Python object (a dataclass is the usual choice), hand it to **Runner.run(..., context=...)**, and the SDK wraps it in a **RunContextWrapper** that every tool in that run receives as its first parameter.

```

import asyncio
from dataclasses import dataclass
from agents import Agent, Runner, RunContextWrapper, function_tool

@dataclass
class UserInfo:
    name: str
    uid: int

@function_tool
async def fetch_user_age(wrapper: RunContextWrapper[UserInfo]) -> str:
    """Returns the age of the user."""
    return f"User {wrapper.context.name} is 47 years old"

async def main():
    user_info = UserInfo(name="Ali", uid=123)

    agent = Agent[UserInfo](
        name="Assistant",
        model=llm_model,
        tools=[fetch_user_age],
    )

    result = await Runner.run(
        starting_agent=agent,
        input="What is the age of the user?",
        context=user_info,
    )
    print(result.final_output)

asyncio.run(main())

```

Note what the prompt does **not** say: it never mentions Ali. The LLM asks for the user's age, the SDK runs your function, and your function reads the name out of `wrapper.context`.

The part that surprises people

The context is **never sent to the LLM**. Look at the schema the SDK generated for that tool:

```

print(fetch_user_age.params_json_schema)
# {'properties': {}, 'title': 'fetch_user_age_args', 'type': 'object', ...}

```

Empty. The `wrapper` parameter is stripped out before the tool description goes to the model, so the model calls `fetch_user_age()` with no arguments and can't see — or invent — the user's identity. Compare that with `calculate_area` in Part 6, where `length` and `width` are in the schema because the model is meant to supply them.

Two rules follow from this:

- Every tool, hook and callback in a single run must use the **same** context type — that's why the agent is written `Agent[UserInfo]`.
- If you want the LLM to *know* something, it belongs in `instructions` or in the input message, not in the context.

Try it: a second tool on the same context

```
@dataclass
class UserInfo:
    name: str
    uid: int
    location: str = "Pakistan"

@function_tool
async def fetch_user_location(wrapper: RunContextWrapper[UserInfo]) -> str:
    """Returns the location of the user."""
    return f"User {wrapper.context.name} is from {wrapper.context.location}"
```

Register both tools, then ask "What is the age of the user, and where are they from?" — one context object, two tools reading different fields out of it, no user data anywhere in the prompt.

Part 8 — Dynamic Instructions: A System Prompt That Changes

Every agent so far had its `instructions` fixed at definition time. The SDK also accepts a `function` there. It runs at the start of each turn and returns the string to use as that turn's system prompt — so the agent's persona can depend on who is asking, what time it is, or how far the conversation has gone.

The core idea

The function takes exactly two parameters — the context wrapper from Part 7 and the agent itself — and returns a `str`.

```

from dataclasses import dataclass
from agents import Agent, Runner, RunContextWrapper

@dataclass
class UserInfo:
    name: str
    uid: int

def special_prompt(ctx: RunContextWrapper[UserInfo], agent: Agent) -> str:
    return (
        f"You are a math expert. User: {ctx.context.name}, Agent: {agent.name}. "
        "Please assist with math-related queries."
    )

math_agent = Agent[UserInfo](
    name="Genius",
    instructions=special_prompt,
    model=llm_model,
)

user = UserInfo(name="Ali", uid=123)
result = Runner.run_sync(math_agent, "What is 17 × 23?", context=user)
print(result.final_output)

```

- `ctx.context` is the object you passed as `context=` in Part 7 — this is where local context and instructions meet.
- `agent` is the agent being run, so `agent.name` and `len(agent.tools)` are available while you build the prompt.
- The function may also be `async def`; the SDK awaits it, so you can hit a database before returning the string.

Note: The callable must accept **exactly two** parameters. Give it one or three and the SDK raises `TypeError: 'instructions' callable must accept exactly 2 arguments (context, agent)`.

Try it: instructions that remember

Any callable object works, not just a function — which is how you keep state between runs:

```

class StatefulInstructions:
    def __init__(self):
        self.count = 0

    def __call__(self, ctx: RunContextWrapper, agent: Agent) -> str:
        self.count += 1
        if self.count == 1:
            return "You are a learning assistant. First interaction – be welcoming."
        return f"You are a learning assistant. Interaction #{self.count} – be brief."

agent = Agent(name="Stateful", instructions=StatefulInstructions(), model=llm_model)

for _ in range(3):
    print(Runner.run_sync(agent, "Tell me about AI in one line").final_output)

```

The first answer greets you; the next two get progressively terser, because the object counted the runs.

What the context does not hold

`RunContextWrapper` carries `.context` (your own object), `.usage` (token counts so far) and `.turn_input`. It does **not** carry a list of past messages. Code like `len(getattr(ctx, "messages", []))` compiles, runs, and quietly returns `0` forever — the branch you wrote for a long conversation never fires. If you want to react to conversation length, count it yourself in your own context object, the way `StatefulInstructions` above does.

Part 9 — Cloning Agents: One Base, Many Variants

Parts 6 and 8 handed you knobs: temperature, tool choice, instructions. Once you have an agent you like, you rarely want to rewrite it from scratch just to turn one of those. `agent.clone(...)` copies an agent and overrides only the fields you name.

The core idea

```
from agents import Agent, Runner, ModelSettings

base_agent = Agent(
    name="BaseAssistant",
    instructions="You are a helpful assistant.",
    model=llm_model,
    model_settings=ModelSettings(temperature=0.7),
)

creative_agent = base_agent.clone(
    name="CreativeAssistant",
    instructions="You are a creative writing assistant. Use vivid language.",
    model_settings=ModelSettings(temperature=0.9),
)

precise_agent = base_agent.clone(
    name="PreciseAssistant",
    instructions="You are a precise, factual assistant.",
    model_settings=ModelSettings(temperature=0.1),
)

for agent in (base_agent, creative_agent, precise_agent):
    print(f"\n{agent.name}:")
    print(Runner.run_sync(agent, "Describe a sunset.").final_output)
```

Anything you don't pass is carried over from the base — neither clone repeats `model=llm_model`, and both still run on Gemini. This is how you build a routing model, a reasoning model and a summariser out of one definition.

The shallow-copy trap

`clone()` is `dataclasses.replace` underneath, which makes a **shallow** copy. A list you don't override isn't copied — the base and the clone hold the *same* list object.

```
base = Agent(name="Base", instructions="Be helpful.", tools=[get_weather])
shared_clone = base.clone(name="SharedClone")

base.tools.append(add)          # appended through the original...
print(len(shared_clone.tools))  # 2 — the clone grew a tool too

independent = base.clone(name="Independent", tools=[*base.tools])
base.tools.append(calculate_area)
print(len(independent.tools))   # 2 — its own list, unaffected
```

The rule: **pass a fresh list whenever you want independence** — `tools=[*base.tools, extra_tool]`. The same applies to `handoffs`, `mcp_servers` and the guardrail lists.

Try it: settings are replaced, not merged

```
from dataclasses import replace

base = Agent(
    name="Base",
    instructions="Be helpful.",
    model_settings=ModelSettings(temperature=0.7, max_tokens=500),
)

careless = base.clone(model_settings=ModelSettings(temperature=0.1))
print(careless.model_settings.max_tokens)    # None – the 500 was dropped

careful = base.clone(model_settings=replace(base.model_settings, temperature=0.1))
print(careful.model_settings.max_tokens)     # 500 – every other field kept
```

A `ModelSettings` handed to `clone()` swaps the whole object, so any field you leave out silently falls back to its default. When you mean to change one setting and keep the rest, `dataclasses.replace` on the base's settings is the safe move.

Recap

Concept	What it gives you
<code>Agent(name, instructions, model, tools)</code>	Defines <i>who</i> the agent is and what it can use
<code>Runner.run_sync / run / run_streamed</code>	Actually executes the agent against an LLM
<code>asyncio</code>	Lets LLM calls (network I/O) run without blocking your whole program
Agent / Run / Global model config	Three scopes for choosing <i>which</i> LLM backs an agent
<code>@function_tool</code>	Turns a Python function into something the agent can call mid-conversation
<code>ModelSettings(temperature, tool_choice, max_tokens)</code>	Tunes <i>how</i> the chosen model behaves on a given run
<code>RunContextWrapper + context=</code>	Passes your own data to tools without ever showing it to the LLM
<code>instructions=<callable></code>	Rebuilds the system prompt on every turn, from context or state
<code>agent.clone(...)</code>	Produces a variant of an agent, overriding only the fields you name

You now have everything needed to build the **First Coded Agent** project: an agent, backed by a provider of your choice, that can reason about a request and reach for a tool when it needs one.

Part 10 — Practice This with Claude Code

Reading code is not the same as building it. This course's approach is to **learn by prompting Claude Code** — Anthropic's agentic coding CLI — and reviewing what it produces, rather than typing everything from scratch or copy-pasting blindly. Open a terminal in your project folder, run **claude**, and work through the prompts below **in order**. Each one builds on the file Claude Code just created or edited for you.

Don't just accept the output — read the diff, ask "why did you do it that way?", and only move to the next prompt once you understand the code you now have.

Prompt-by-part

Part	Prompt to give Claude Code
0. Setup	Scaffold a uv-managed Python project called hello-agent with openai-agents and python-dotenv as dependencies. Create a .env file with empty OPENAI_API_KEY and GEMINI_API_KEY entries, and make sure .env is gitignored.
1. OpenAI key	Create main.py using the OpenAI Agents SDK: an Agent named Assistant with instructions "You are a helpful assistant", run synchronously against the prompt "Write a haiku about recursion in programming." Load OPENAI_API_KEY from .env. Then explain what Agent and Runner.run_sync each do.
2. Gemini key	Modify main.py so the same Agent uses Gemini's gemini-2.5-flash model instead of OpenAI's default – wire up AsyncOpenAI with Gemini's OpenAI-compatible base_url and wrap it in OpenAIChatCompletionsModel. Explain line-by-line why swapping the provider didn't require touching Agent or Runner.
3. Runner & asyncio	Rewrite main.py to call Runner.run inside an async def main() instead of run_sync, executed via asyncio.run(). Then add a second version that fires two different prompts at the same agent concurrently with asyncio.gather and prints both results. Explain why the concurrent version is faster.
4. Model configuration	Refactor my agent so the Gemini model is set at the Run level via RunConfig instead of the Agent level. Show me the same agent being called once with the Gemini run_config and once without one, so I can see the difference. Explain when I'd pick Run-level over Agent-level.
5. Tool use	Add a @function_tool called get_weather(city: str) -> str returning a hardcoded string, and a second tool add(a: int, b: int) -> int. Wire both into the agent's tools list and run it against "What's 42 plus 58, and what's the weather in Lahore?" Then walk me through, step by step, what happened between my prompt and the final answer, including which tools got called and why.
6. Model settings	Take my weather-and-add agent and run the same question three times with model_settings=ModelSettings(tool_choice="auto"), then "required", then "none". Print all three answers side by side and explain what changed and why the "none" answer is different.
7. Local context	Add a UserInfo dataclass with name and uid, pass it to Runner.run via context=, and add a tool that reads wrapper.context.name. Then print the tool's params_json_schema and explain why the wrapper parameter isn't in it.
8. Dynamic instructions	Replace my agent's instructions string with a function taking (context, agent) that greets the user by name from context and mentions the agent's own name. Print the resolved system prompt before the model call, then convert the function into a callable class that counts interactions.

Part	Prompt to give Claude Code
9. Agent cloning	Create a base agent with temperature 0.7 and one tool, then clone it into a creative variant and a precise variant. Prove to me with is-comparisons which attributes are shared between base and clone and which are not.

Debug-it prompts (learn by breaking things)

Understanding deepens when you have to *fix* something, not just read it. Try these after finishing Part 5:

- Deliberately break my Gemini setup by using the wrong `base_url`, show me the exact error it raises, then walk me through how you diagnosed and fixed it.
- Rename the `get_weather` function to `fetch_weather` but leave the docstring and tools list referencing the old name. Run it, show me what breaks, then fix it and explain why the SDK cares about the function name at all.
- Remove the type hint from the `city` parameter in `get_weather`. Run the agent again – does it still work? Explain what the SDK does differently when a tool parameter has no type hint.
- Change my dynamic instructions function so it takes only the context parameter, run it, and show me the exact `TypeError` the SDK raises before fixing it back.
- Take my local-context example and remove `context=user_info` from the `Runner.run` call, leaving the tool untouched. Run it, show me the exact error, and explain which line of the tool blew up and why.

Capstone prompt

Once you've been through all ten parts, try building something new end-to-end in one shot, then critique what comes back:

```
Using the OpenAI Agents SDK, build a small CLI agent powered by Gemini
(Agent-level configuration) with two tools of your own choosing (not
get_weather or add). Expose it through an async main() run via
asyncio.run(). Then give me three test prompts that demonstrate the
agent choosing between the two tools correctly, and explain the tool-
selection reasoning for each one.
```

Compare what Claude Code builds against Parts 1–9 of this guide — if you can explain every line without asking "why," you've learned the material.